医療・福祉BPO詳細設計

スコープ: 介護施設（通所介護/訪問介護/特養/老健/グループホーム）+ 診療所（無床〜19床）
事業所数: 約37万（介護14万 + クリニック10万 + 福祉系13万）
市場規模: 約59兆円/年
BPO月額: ¥13-30万（BPOコア¥250,000 + 追加モジュール¥100,000/個）
ゲノム系統: 歯科系（保険請求+書類地獄+人手不足の同構造）
パイロット: 通所介護（デイサービス）10-50名規模 + 内科クリニック3-10名規模
詳細設計: e_04（介護福祉）、e_07（医療クリニック）を統合し、8モジュール全ての実装仕様を記載
既存実装: `workers/bpo/nursing/pipelines/care_billing_pipeline.py` （5ステップ）、 `workers/bpo/clinic/pipelines/medical_receipt_pipeline.py` （6ステップ）

---

0. エグゼクティブサマリー

なぜ医療・福祉BPOか

```
医療・福祉の中小事業所は：
  ・介護報酬請求の返戻率3-5%。月500万円請求で年120万円の入金遅延
  ・レセプト点検に院長が毎月5-10時間。時給換算で年120-240万円の機会損失
  ・ケアマネ1人が30件抱えて限界。ケアプラン作成が属人化
  ・夜勤シフト管理が地獄。人員配置基準違反=指定取消のリスク
  ・実地指導（行政監査）の書類準備で毎年数日消える
  ・介護記録・日誌が手書き or 二重入力
  ・服薬管理の抜け漏れが事故につながる
  ・未収金管理が属人化。公費適用判定に詳しい事務員が退職=崩壊

 → 全部AIで代行できる = 年間500-1,000万円の価値
 → 歯科レセプトエンジンの横展開で最も効率的に立ち上がる
```

8モジュール全体像

```
┌──────────────────────────────────────────────────────────────┐
│       医療・福祉BPO（BPOコア¥250,000 + 追加モジュール¥100,000/個）        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ★ Phase A（Week 1–2）: 最優先                                 │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ① 介護報酬請求AI    │  │ ② レセプト点検AI              │  │
│  │ 実績→請求→加算→国保連│  │ 病名×処置×薬剤チェック       │  │
│  │ ¥5–10万/月の価値    │  │ ¥5–15万/月の価値              │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase B（Week 3）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ③ ケアプラン作成支援 │  │ ④ 勤怠・シフト管理            │  │
│  │ アセスメント→計画生成 │  │ 夜勤ルール+配置基準チェック   │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase C（Week 4）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑤ 記録・日誌AI      │  │ ⑥ 服薬管理                   │  │
│  │ SOAP記録自動生成    │  │ 相互作用チェック+アラート     │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  Phase D（Week 5）:                                            │
│  ┌──────────────────┐  ┌──────────────────────────┐          │
│  │ ⑦ 監査・実地指導準備 │  │ ⑧ 請求・入金管理              │  │
│  │ チェックリスト+書類管理│  │ 自己負担金+未収金+公費適用   │  │
│  └──────────────────┘  └──────────────────────────┘          │
│                                                              │
│  横断:                                                         │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ SaaS連携層（カイポケ CSV / ほのぼの CSV / ORCA連携）    ||  │
│  └──────────────────────────────────────────────────────┘    │
│  ┌──────────────────────────────────────────────────────┐    │
│  │ 算定ルールDB（介護報酬単位マスタ + 診療報酬点数マスタ）  ||  │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

#	モジュール	対象	Tier	月額価値	Phase	実装場所
1	介護報酬請求AI	介護施設	★キラー	¥5-10万	A	`workers/bpo/nursing/pipelines/care_billing_pipeline.py`
2	レセプト点検AI	クリニック	★キラー	¥5-15万	A	`workers/bpo/clinic/pipelines/medical_receipt_pipeline.py`
3	ケアプラン作成支援	介護施設	Tier1	¥3-5万	B	`workers/bpo/nursing/pipelines/care_plan_pipeline.py`
4	勤怠・シフト管理	共通	Tier1	¥3-5万	B	`workers/bpo/common/pipelines/shift_pipeline.py`
5	記録・日誌AI	介護施設	Tier2	¥2-3万	C	`workers/bpo/nursing/pipelines/care_record_pipeline.py`
6	服薬管理	共通	Tier2	¥2-3万	C	`workers/bpo/common/pipelines/medication_pipeline.py`
7	監査・実地指導準備	共通	Tier2	¥2-3万	D	`workers/bpo/common/pipelines/audit_prep_pipeline.py`
8	請求・入金管理	共通	Tier2	¥2-3万	D	`workers/bpo/common/pipelines/billing_collection_pipeline.py`

---

1. 介護報酬請求AI（★キラーフィーチャー / 介護施設向け）

1.1 課題（金額換算の価値）

```
■ 直接効果①：返戻・過誤の削減
  月間請求額500万円 × 返戻率2%改善 = 月10万円 = 年120万円の入金安定化

■ 直接効果②：加算の取り漏れ防止
  処遇改善加算I（加算率13.7%）の計上漏れが月あり
  月間500万円 × 13.7% × 漏れ率5% = 月3.4万円 = 年41万円
  特定事業所加算、夜勤体制加算等を含めると年50-100万円

■ 直接効果③：事務員の請求業務時間削減
  月40-60時間 → 自動チェックで50%削減 = 月20-30時間削減
  時給1,500円 × 25時間 = 月3.75万円 = 年45万円

■ 合計効果：年206-265万円
  → BPO月5万円（年60万円）で ROI 3.4〜4.4倍
```

1.2 業務フロー

```
【日次】
  サービス提供
    → 実績記録（介護ソフトに入力 or 紙記録）
    → 利用者の要介護認定有効期間の自動確認
    → ★AI: 実績データのリアルタイムバリデーション

【月初（1〜5日）】
  前月分の実績データ集計
    → ★AI Step 1: record_reader（実績記録データ取得）
    → ★AI Step 2: service_code_mapper（サービスコード→報酬単位数マッピング）
    → ★AI Step 3: limit_checker（区分支給限度額チェック）
    → ★AI Step 4: addition_checker（加算チェック）
    → ★AI Step 5: deduction_checker（減算チェック）
    → ★AI Step 6: compliance_checker（コンプライアンスチェック）
    → ★AI Step 7: output_generator（国保連請求データ生成）
    → 事務員が最終確認→承認

【毎月10日（法定期限: 介護保険法施行規則第30条）】
  国保連に電子請求（伝送）

【翌月〜翌々月】
  審査結果確認 → 返戻・過誤の対応
  → ★AI: 返戻原因分析→再請求データ自動生成
```

1.3 AI化ポイント

```
Level 1: ルールベースチェック（実装済み）
  - サービスコード×要介護度の整合性チェック
  - 区分支給限度基準額の超過チェック
  - 加算算定要件の充足チェック（処遇改善加算I〜V、特定処遇改善加算I〜II）
  - サービス提供日と請求対象期間の一致確認

Level 2: AI予測チェック（Phase 2）
  - 過去の返戻パターンから返戻リスクを予測
  - 加算の取り漏れを過去実績から提案
  - 類似事業所との比較分析（匿名化データ）

Level 3: 完全自動請求（Phase 3）
  - 実績記録→請求データ生成→国保連伝送まで自動化
  - 返戻率0.5%以下を達成後に段階的に自律度を上げる
```

1.4 パイプラインステップ定義

```
実装: workers/bpo/nursing/pipelines/care_billing_pipeline.py
クラス: CareBillingPipeline
エントリ: run_care_billing_pipeline(company_id, input_data)
```

Step 1: record_reader（実績記録データ取得）

項目	内容
マイクロエージェント	`micro/record_reader`
入力	`service_records[]`（JSON直渡し or CSV/Excel or 介護ソフトCSVエクスポート）
処理	レコード正規化、必須フィールド補完、日付パース
出力	`list[dict]`（user_id, user_name, service_type, care_level, service_date, service_hours, service_code, units）
エラー条件	`service_records` が空 → ValueError

Step 2: service_code_mapper（サービスコード→報酬単位数マッピング）

項目	内容
マイクロエージェント	`micro/service_code_mapper`
入力	Step 1の正規化レコード
処理	サービスコードマスタ参照→単位数付与。未設定時はサービス種類×時間から推計
出力	`list[dict]`（units フィールドが付与されたレコード）
計算ロジック	下記「1.5 計算ロジック」参照

サービスコード→単位数の推計ロジック（フォールバック）

```
# 訪問介護（身体介護中心）
if service_type == "訪問介護":
    if hours < 0.5:   units = 167   # 20分以上30分未満
    elif hours < 1.0: units = 245   # 30分以上1時間未満
    elif hours < 1.5: units = 388   # 1時間以上1時間30分未満
    elif hours < 2.0: units = 452   # 1時間30分以上2時間未満
    else:             units = 579   # 2時間以上

# 通所介護（要介護3、通常規模型の例）
if service_type == "通所介護":
    if hours < 3.0:   units = 386   # 3時間未満
    elif hours < 5.0: units = 561   # 3時間以上5時間未満
    elif hours < 7.0: units = 738   # 5時間以上7時間未満
    else:             units = 867   # 7時間以上
```

Step 3: limit_checker（区分支給限度額チェック）

項目	内容
マイクロエージェント	`micro/limit_checker`
入力	Step 2のレコード + 利用者別の当月累計単位数
処理	要介護度別の区分支給限度基準額との照合
出力	超過警告リスト + 自己負担案内対象リスト
法令根拠	介護保険法第43条（区分支給限度基準額）

区分支給限度基準額（令和6年度）

要介護度	限度額（単位/月）	金額目安（1単位10円）
要支援1	5,032	約50,320円
要支援2	10,531	約105,310円
要介護1	16,765	約167,650円
要介護2	19,705	約197,050円
要介護3	27,048	約270,480円
要介護4	30,938	約309,380円
要介護5	36,217	約362,170円

```
# 超過判定ロジック
CARE_LEVEL_LIMIT_UNITS = {
    "要支援1": 5_032, "要支援2": 10_531,
    1: 16_765, 2: 19_705, 3: 27_048, 4: 30_938, 5: 36_217,
}

user_monthly_total = sum(r["units"] for r in user_records)
limit = CARE_LEVEL_LIMIT_UNITS[care_level]
if user_monthly_total > limit:
    excess_units = user_monthly_total - limit
    # 超過分は保険給付対象外（全額自己負担）
    # 介護保険法第43条第1項
    warnings.append(f"限度額超過: {excess_units}単位（全額自己負担）")
```

Step 4: addition_checker（加算チェック）

項目	内容
マイクロエージェント	`micro/addition_checker`
入力	Step 2のレコード + 事業所情報（加算レベル、地域区分、体制情報）
処理	加算率の適用→加算単位数・加算金額の算出
出力	`(base_units, addition_units, addition_amount_yen)`
法令根拠	介護保険法第41条第4項（介護報酬の算定基準）、厚生労働省告示第19号

処遇改善加算の計算式

```
# 令和6年度（2024年6月～）処遇改善加算 新体系
# ※旧・処遇改善加算 + 特定処遇改善加算 + ベースアップ等支援加算を一本化

# 訪問介護の場合の加算率
SHOGUU_KAIZEN_RATES_HOUKAN = {
    "I":   0.245,  # 加算I（旧I+旧特定I+旧ベースアップ）
    "II":  0.224,  # 加算II
    "III": 0.182,  # 加算III
    "IV":  0.137,  # 加算IV（旧処遇改善加算Iに相当）
    "V1":  0.100,  # 加算V(1)
    "V2":  0.055,  # 加算V(2)
}

# 通所介護の場合の加算率
SHOGUU_KAIZEN_RATES_TSUSHO = {
    "I":   0.094,
    "II":  0.086,
    "III": 0.070,
    "IV":  0.055,
    "V1":  0.040,
    "V2":  0.022,
}

# 加算額計算
# 加算単位数 = 対象サービスの総単位数 × 加算率
# 加算金額（円） = 加算単位数 × 地域単価
addition_units = int(base_units * rate)
addition_amount = int(addition_units * region_unit_price)
```

地域区分別の単位数単価（1単位あたり）

```
REGION_UNIT_PRICE = {
    "1級地": 11.40,   # 東京23区
    "2級地": 11.12,   # 東京都特別区の一部
    "3級地": 11.05,   # 大阪市、横浜市等
    "4級地": 10.84,   # 名古屋市、京都市等
    "5級地": 10.70,   # 政令指定都市等
    "6級地": 10.42,   # 中核市等
    "7級地": 10.21,   # 一般市等
    "その他": 10.00,  # その他地域
}
```

その他の主要加算と算定要件

加算名	加算単位（例）	算定要件	法令根拠
特定事業所加算I	所定単位数×20%	全職員の20%以上が介護福祉士+会議月1回以上+個別計画作成	告示19号別表1注7
特定事業所加算II	所定単位数×10%	介護福祉士30%以上+会議月1回以上	同上
夜勤体制加算	24単位/日	夜勤職員が最低基準+1名以上	告示19号別表8注13
ADL維持等加算I	30単位/月	要介護3以上が15%以上+ADL利得が0以上	告示19号別表6注14
認知症加算	60単位/日	日常生活自立度III以上+専門研修修了職員配置	告示19号別表6注10
入浴介助加算I	40単位/日	入浴介助を行った場合	告示19号別表6注8
入浴介助加算II	55単位/日	医師等が入浴計画作成+個別入浴介助	同上
送迎加算	片道47単位	送迎を行った場合（通所介護）	告示19号別表6注3

Step 5: deduction_checker（減算チェック）

項目	内容
マイクロエージェント	`micro/deduction_checker`
入力	事業所の人員配置情報 + 定員情報
処理	減算条件の該当チェック→減算率の適用
出力	減算警告リスト + 減算後単位数
法令根拠	介護保険法第73条（指定の基準）、人員基準省令

主な減算ルール

```
DEDUCTION_RULES = {
    "人員基準欠如減算": {
        "条件": "配置基準を満たしていない場合",
        "減算率": 0.70,  # 所定単位数の70%に減算
        "法令": "指定居宅サービス等の事業の人員、設備及び運営に関する基準 第93条",
        "判定ロジック": """
            通所介護の場合:
              利用者15名まで → 介護職員1名以上
              利用者15名超   → 15名超の5名ごとに1名追加
              看護職員       → 利用定員10名超で1名以上
            訪問介護の場合:
              常勤のサービス提供責任者 → 利用者40名ごとに1名以上
        """,
    },
    "定員超過減算": {
        "条件": "利用定員の105%超過の場合",
        "減算率": 0.70,  # 所定単位数の70%に減算
        "法令": "指定居宅サービス等の事業の人員、設備及び運営に関する基準 第93条",
        "判定ロジック": """
            月平均利用者数 > 利用定員 × 1.05 → 減算適用
            日ごとの判定: 当日利用者数 > 利用定員 × 1.05 → 当日分減算
        """,
    },
    "身体拘束減算": {
        "条件": "身体拘束の適正化未実施",
        "減算率": 0.90,  # 所定単位数の90%に減算（10%減算）
        "法令": "指定介護老人福祉施設の人員、設備及び運営に関する基準 第11条第5項",
    },
}

# 判定ロジック
def check_staff_shortage(facility_info: dict) -> float:
    """人員配置基準の充足チェック→減算率を返す"""
    service_type = facility_info["service_type"]
    if service_type == "通所介護":
        capacity = facility_info["capacity"]
        care_staff = facility_info["care_staff_count"]
        required = 1 if capacity <= 15 else 1 + ((capacity - 15) // 5 + 1)
        if care_staff < required:
            return 0.70  # 人員基準欠如減算
    return 1.00  # 減算なし
```

Step 6: compliance_checker（コンプライアンスチェック）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	全ステップのデータ
処理	返戻リスク判定（被保険者資格確認、ケアプラン整合性、公費番号確認）
出力	返戻リスクリスト（リスクレベル: HIGH/MEDIUM/LOW）
法令根拠	介護保険法第27条（要介護認定）、第41条（居宅介護サービス費の支給）

返戻リスク判定ルール

```
REJECTION_RISK_RULES = [
    {
        "id": "R001",
        "name": "被保険者資格不一致",
        "level": "HIGH",
        "check": "要介護認定有効期間内にサービス提供日が含まれるか",
        "法令": "介護保険法第27条（要介護認定の有効期間）",
    },
    {
        "id": "R002",
        "name": "サービスコード不一致",
        "level": "HIGH",
        "check": "サービスコードの先頭2桁がサービス区分と一致するか",
        "法令": "介護給付費単位数表 別表1～11",
    },
    {
        "id": "R003",
        "name": "ケアプラン未作成",
        "level": "HIGH",
        "check": "サービス提供日にケアプランが有効であるか",
        "法令": "介護保険法第41条第6項（居宅サービス計画に基づく提供）",
    },
    {
        "id": "R004",
        "name": "公費負担者番号不一致",
        "level": "MEDIUM",
        "check": "公費適用がある場合、負担者番号が有効か",
        "法令": "生活保護法第15条の2（介護扶助）",
    },
    {
        "id": "R005",
        "name": "サービス提供日と請求月の不一致",
        "level": "HIGH",
        "check": "サービス提供日の年月が請求対象期間と一致するか",
        "法令": "介護保険法施行規則第30条",
    },
    {
        "id": "R006",
        "name": "加算算定要件不備",
        "level": "MEDIUM",
        "check": "処遇改善計画書の提出済み確認、実績報告の提出期限確認",
        "法令": "厚生労働省告示第19号（介護報酬の算定基準）",
    },
]
```

Step 7: output_generator（国保連請求データ生成）

項目	内容
マイクロエージェント	`micro/output_generator`
入力	全ステップの処理結果
処理	国保連伝送用CSVデータの生成（介護給付費明細書様式）
出力	`CareBillingPipelineResult`（請求総額、加算額、返戻リスク、レコード一覧）
法令根拠	介護保険法施行規則第30条（請求書の提出）

1.5 計算ロジック

介護報酬の算出フロー

```
① 基本報酬（単位数）の決定
   サービス種類 × 要介護度 × 提供時間 → 基本単位数
   例: 通所介護/要介護3/7時間以上8時間未満 = 896単位

② 加算の適用
   基本単位数 + 各種加算単位数 = 総単位数
   例: 896 + 入浴40 + 送迎47×2 + 個別機能訓練56 = 1,086単位

③ 減算の適用
   減算条件に該当する場合、所定単位数 × 減算率
   例: 人員基準欠如の場合 → 1,086 × 0.70 = 760単位

④ 処遇改善加算
   対象サービスの総単位数 × 加算率
   例: 760 × 0.094（通所介護 加算I）= 71単位

⑤ 金額換算
   (総単位数 + 処遇改善加算単位数) × 地域単価
   例: (760 + 71) × 10.84（4級地）= 9,004円

⑥ 利用者負担
   金額 × 負担割合（1割/2割/3割）
   例: 9,004 × 0.1 = 901円（利用者負担）
       9,004 − 901 = 8,103円（保険給付額 → 国保連請求）

⑦ 公費適用（生活保護等）
   生活保護受給者の場合、利用者負担分も公費から支給
   法令: 生活保護法第15条の2
```

1.6 データモデル

```
@dataclass
class ServiceRecordResult:
    """処理済みサービス実績の1件"""
    user_id: str              # 被保険者番号
    user_name: str            # 利用者名
    service_type: str         # サービス区分（訪問介護/通所介護等）
    care_level: int           # 要介護度（1〜5）
    service_date: str         # サービス提供日（YYYY-MM-DD）
    service_code: str         # サービスコード（6桁）
    units: int                # 基本単位数
    addition_units: int       # 加算単位数
    deduction_rate: float     # 減算率（1.0 = 減算なし）
    total_units: int          # 算定単位数（units × deduction_rate + addition_units）
    amount: int               # 金額（円）
    copay_ratio: float        # 利用者負担割合（0.1/0.2/0.3）
    copay_amount: int         # 利用者負担額（円）
    insurance_amount: int     # 保険給付額（円）
    public_expense_code: str  # 公費負担者番号（該当なしは空文字）


@dataclass
class CareBillingPipelineResult:
    """介護報酬請求パイプラインの最終結果"""
    success: bool
    period_year: int                          # 請求対象年
    period_month: int                         # 請求対象月
    facility_type: str                        # 事業所種類
    facility_code: str                        # 事業所番号（10桁）
    region: str                               # 地域区分
    shoguu_kaizen_level: str                  # 処遇改善加算レベル
    records: list[ServiceRecordResult]        # 請求レコード一覧
    total_units: int                          # 総単位数
    base_units: int                           # 基本単位数合計
    addition_units: int                       # 加算単位数合計
    total_amount: int                         # 請求総額（円）
    insurance_amount: int                     # 保険給付額合計（円）
    copay_amount: int                         # 利用者負担額合計（円）
    rejection_risks: list[dict]               # 返戻リスク
    deduction_warnings: list[str]             # 減算警告
    warnings: list[str]                       # その他警告
    steps_completed: list[str]                # 完了ステップ
    error: str | None = None
```

入力データ JSON構造

```
{
  "service_records": [
    {
      "user_id": "U001",
      "user_name": "鈴木一郎",
      "insured_number": "0123456789",
      "service_type": "通所介護",
      "care_level": 3,
      "care_level_valid_from": "2025-10-01",
      "care_level_valid_to": "2026-09-30",
      "service_date": "2026-03-05",
      "service_hours": 7.5,
      "service_code": "151131",
      "units": 896,
      "additions": ["入浴介助加算I", "送迎加算"],
      "copay_ratio": 0.1,
      "public_expense_code": ""
    }
  ],
  "facility_info": {
    "facility_code": "1370100001",
    "facility_type": "通所介護",
    "region": "4級地",
    "shoguu_kaizen_level": "I",
    "capacity": 30,
    "care_staff_count": 5,
    "nurse_count": 1
  },
  "period_year": 2026,
  "period_month": 3
}
```


1.7 法令・規格準拠ルール

法令	条文	適用箇所
介護保険法 第27条	要介護認定の有効期間	被保険者資格チェック
介護保険法 第41条	居宅介護サービス費の支給	保険給付の算定基準
介護保険法 第43条	区分支給限度基準額	限度額超過チェック
介護保険法 第73条	指定居宅サービス事業者の基準	人員配置基準
介護保険法施行規則 第30条	請求書の提出	請求期限（毎月10日）
厚生労働省告示第19号	指定居宅サービスに要する費用の額の算定に関する基準	全算定ルール
人員基準省令	各サービス種類の配置基準	人員欠如減算
生活保護法 第15条の2	介護扶助	公費負担判定
個人情報保護法 第2条第3項	要配慮個人情報	介護度・病歴の取り扱い

1.8 制約・前提条件

- 介護報酬単位数表はゲノムJSONとして管理。3年ごとの報酬改定時にJSON更新
- 国保連への電子請求（伝送）はPhase 1ではCSV出力まで。伝送APIはPhase 2+
- 要介護認定情報は利用者マスタとして事前登録が必要
- 公費（生活保護等）の適用判定は受給者証番号の照合のみ。自動判定はPhase 2+

1.9 独自価値（競合にない差別化）

- **加算取り漏れの能動提案**: 既存ソフトは「入力した加算のチェック」のみ。シャチョツーは「取れるのに取っていない加算」を提案する
- **返戻予測**: 過去の返戻パターンからリスクスコアを算出。返戻前に修正を促す
- **学習する算定エンジン**: 使うほど事業所固有の算定パターンを学習。2回目以降は自動候補の精度が上がる

---

2. レセプト点検AI（★キラーフィーチャー / クリニック向け）

2.1 課題（金額換算の価値）

```
■ 直接効果①：査定損失の回避
  月間請求額1,000万円 × 査定率0.3%改善 = 月3万円 = 年36万円
  月間請求額2,000万円の医院なら = 月6万円 = 年72万円

■ 直接効果②：算定漏れの防止
  特定疾患療養管理料（225点×2回=4,500円/患者/月）の算定漏れ
  月20人分の漏れ → 月9万円 = 年108万円
  外来管理加算、在宅時医学総合管理料等を含めると年36-120万円

■ 直接効果③：院長のレセプトチェック時間削減
  月5-10時間 × 時間単価20,000円 = 月10-20万円 = 年120-240万円

■ 合計効果：年192-432万円
  → BPO月8万円（年96万円）で ROI 2.0〜4.5倍
```

2.2 業務フロー

```
【日次】
  診療 → カルテ入力 → レセコン入力 → 会計
  → ★AI: リアルタイム算定チェック（カルテ入力時に警告表示）← Phase 2+

【月初（1〜10日)】
  前月分レセプト一括出力
    → ★AI Step 1: receipt_parser（レセプトデータ取得・正規化）
    → ★AI Step 2: disease_check（傷病名×診療行為の整合性チェック）
    → ★AI Step 3: medication_check（薬剤×傷病名の適応チェック）
    → ★AI Step 4: scoring（算定ルール・回数制限チェック）
    → ★AI Step 5: report（診療科別レポート生成）
    → ★AI Step 6: validator（最終バリデーション）
    → 院長が最終チェック→承認

【毎月10日（法定期限: 健康保険法施行規則第47条)】
  オンライン請求（支払基金・国保連）

【翌月〜翌々月】
  返戻・査定の確認→再請求
  → ★AI: 査定パターン分析→再発防止提案
```

2.3 AI化ポイント

```
Level 1: ルールベースチェック（実装済み）
  - 傷病名と診療行為の整合性チェック（算定対象傷病名の有無）
  - 薬剤と傷病名の適応チェック（添付文書の適応症との照合）
  - 算定回数上限チェック（月1回、月2回等の制限）
  - 併算定不可ルールチェック（同時算定不可の組み合わせ）

Level 2: AI予測チェック（Phase 2）
  - 過去の査定パターンから査定リスクを予測
  - 算定漏れを過去実績から提案（「この患者に管理料が算定可能です」）
  - 支払基金の審査傾向を地域別に分析

Level 3: リアルタイムチェック（Phase 3）
  - カルテ入力時にリアルタイムで算定チェック
  - 電子カルテとのAPI連携（ORCA、メディコム等）
```

2.4 パイプラインステップ定義

```
実装: workers/bpo/clinic/pipelines/medical_receipt_pipeline.py
クラス: MedicalReceiptPipeline
エントリ: run_medical_receipt_pipeline(company_id, input_data, task)
```

Step 1: receipt_parser（レセプトデータ取得・正規化）

項目	内容
マイクロエージェント	`micro/receipt_parser`
入力	`receipts[]` （JSON直渡し or レセプトテキスト or レセコンCSV）
処理	正規化（patient_id, department, month, procedures[], medications[], diseases[]）
出力	`list[dict]` 正規化済みレセプトデータ
エラー条件	receipts/textいずれも未設定 → エラー

Step 2: disease_check（傷病名×診療行為の整合性チェック）

項目	内容
マイクロエージェント	`micro/disease_checker`
入力	Step 1のレセプトデータ
処理	診療科別の管理料→対象傷病名の照合、高査定リスク行為の検出
出力	`StepResult` （findings: error/warning/info）
法令根拠	健康保険法第76条（療養の給付に関する費用の算定方法）、診療報酬点数表

診療科別の主要チェックルール

```
MEDICAL_RULES = {
    "内科": {
        "management_fees": {
            "特定疾患療養管理料": {
                "max_per_month": 2,
                "points": 225,  # 225点×2=450点（4,500円）/月
                "target_diseases": [
                    "糖尿病", "高血圧症", "脂質異常症", "喘息",
                    "慢性気管支炎", "甲状腺機能障害", "てんかん",
                    "慢性腎不全", "悪性新生物", "肝疾患",
                    # 厚生労働大臣が定める疾患（告示別表第1）
                ],
                "exclusion": ["生活習慣病管理料"],  # 併算定不可
                "法令": "診療報酬点数表 B000-4",
            },
            "生活習慣病管理料I": {
                "max_per_month": 1,
                "points": 610,  # 610点（6,100円）/月
                "target_diseases": ["糖尿病", "高血圧症", "脂質異常症"],
                "exclusion": ["特定疾患療養管理料"],
                "法令": "診療報酬点数表 B001-3-3",
            },
            "外来管理加算": {
                "max_per_month": 1,  # 同一日に処置・手術がない再診
                "points": 52,
                "conditions": "処置・手術・麻酔を行わない再診時",
                "法令": "診療報酬点数表 A002注8",
            },
        },
    },
    "整形外科": {
        "rehabilitation": {
            "運動器リハビリテーション料I": {
                "points_per_unit": 185,
                "max_days": 150,  # 算定日数上限
                "standard_sessions": 3,  # 1日あたり上限単位数
                "target_diseases": [
                    "骨折", "関節拘縮", "腰痛症", "変形性関節症",
                    "脊柱管狭窄症", "関節リウマチ",
                ],
                "法令": "診療報酬点数表 H002",
            },
        },
    },
    "皮膚科": {
        "management_fees": {
            "皮膚科特定疾患指導管理料I": {
                "max_per_month": 1,
                "points": 250,
                "target_diseases": ["アトピー性皮膚炎", "帯状疱疹", "尋常性乾癬"],
                "法令": "診療報酬点数表 B001-8",
            },
        },
    },
    "眼科": {
        "examinations": {
            "精密眼底検査": {
                "max_per_month": 1,
                "points": 56,
                "法令": "診療報酬点数表 D255",
            },
            "精密眼圧測定": {
                "max_per_month": 1,
```

```
                "points": 82,
                "法令": "診療報酬点数表 D264",
            },
        },
    },
}
```

Step 3: medication_check（薬剤×傷病名の適応チェック）

項目	内容
マイクロエージェント	`micro/medication_checker`
入力	Step 1のレセプトデータ
処理	薬剤→適応傷病名の照合、投薬日数チェック（長期処方の検出）
出力	`StepResult`（findings: error/warning/info）
法令根拠	薬機法第14条（医薬品の承認）、医薬品添付文書

薬剤適応チェックルール（主要薬剤）

```
MEDICATION_DISEASE_RULES = {
    # 降圧薬
    "アムロジピン": ["高血圧症", "狭心症"],
    "カンデサルタン": ["高血圧症", "慢性心不全"],
    "エナラプリル": ["高血圧症", "慢性心不全"],
    # 糖尿病薬
    "メトホルミン": ["2型糖尿病"],
    "シタグリプチン": ["2型糖尿病"],
    "エンパグリフロジン": ["2型糖尿病", "慢性心不全"],
    # 脂質異常症
    "アトルバスタチン": ["高コレステロール血症", "家族性高コレステロール血症"],
    "ロスバスタチン": ["高コレステロール血症", "家族性高コレステロール血症"],
    # 消化器
    "ランソプラゾール": ["胃潰瘍", "逆流性食道炎", "NSAID潰瘍予防"],
    "ラベプラゾール": ["胃潰瘍", "十二指腸潰瘍", "逆流性食道炎"],
    # NSAIDs
    "ロキソプロフェン": ["疼痛", "炎症", "関節リウマチ", "変形性関節症"],
    "セレコキシブ": ["関節リウマチ", "変形性関節症", "腰痛症"],
    # 抗菌薬
    "アモキシシリン": ["呼吸器感染症", "尿路感染症", "ヘリコバクター・ピロリ感染症"],
    "レボフロキサシン": ["呼吸器感染症", "尿路感染症", "皮膚感染症"],
}

# 投薬日数制限（新薬等）
MAX_PRESCRIPTION_DAYS = {
    "デフォルト": 90,         # 長期処方の警告閾値
    "新医薬品": 14,          # 新薬は原則14日まで（薬担規則第20条）
    "麻薬": 30,              # 麻薬は30日まで
    "向精神薬（第1種）": 14,  # ベンゾジアゼピン系は14日〜30日
}
```

Step 4: scoring（算定ルール・回数制限チェック）

項目	内容
マイクロエージェント	`micro/scoring`
入力	Step 1のレセプトデータ
処理	算定回数上限チェック、併算定不可チェック、リスクスコア算出
出力	`StepResult`（risk_points_by_patient, findings）
計算ロジック	`リスク点数 = 超過算定点数 × POINTS_TO_YEN（1点=10円）`

主な回数制限ルール

```
FREQUENCY_LIMITS = {
    # 医学管理等
    "特定疾患療養管理料": 2,       # 月2回まで
    "外来管理加算": 1,             # 再診のつど（処置なし時のみ）
    "生活習慣病管理料": 1,         # 月1回まで
    "在宅時医学総合管理料": 1,     # 月1回まで
    "特定薬剤治療管理料": 1,       # 月1回まで
    "悪性腫瘍特異物質治療管理料": 1, # 月1回まで
    # 検査
    "HbA1c": 1,                    # 月1回まで（DM管理）
    "心電図検査": 1,               # 月1回まで（原則）
    "精密眼底検査": 1,             # 月1回まで
}

# 併算定不可ルール
EXCLUSION_RULES = [
    ("特定疾患療養管理料", "生活習慣病管理料"),  # 同一月に併算定不可
    ("外来管理加算", "処置料"),                  # 処置実施日は算定不可
    ("初診料", "再診料"),                        # 同一日に併算定不可
]

# リスクスコア計算
def calculate_risk_score(receipt: dict) -> int:
    """査定リスクスコアの算出（点数）"""
    risk_points = 0
    for proc_name, max_count in FREQUENCY_LIMITS.items():
        actual = count_procedures(receipt, proc_name)
        if actual > max_count:
            excess = actual - max_count
            points_per = get_points(proc_name)
            risk_points += points_per * excess
    return risk_points  # 点数 → ×10で円換算
```

Step 5: report（診療科別レポート生成）

項目	内容
マイクロエージェント	`micro/report_generator`
入力	全findingsデータ + レセプトデータ
処理	診療科別集計、エラー/警告カウント、トップ5問題点抽出
出力	`StepResult`（report: JSON）

Step 6: validator（最終バリデーション）

項目	内容
マイクロエージェント	`micro/validator`
入力	全ステップの結果
処理	必須フィールド確認、レセプト0件チェック、返戻リスクの有無判定
出力	`StepResult` （has_rejection_risk: bool）

2.5 計算ロジック

診療報酬の算出フロー

```
① 基本診療料の算定
   初診料: 291点（2,910円）
   再診料: 75点（750円）
   外来管理加算: 52点（520円）← 処置なし再診時のみ

② 医学管理等
   特定疾患療養管理料: 225点 × 月2回まで
   生活習慣病管理料: 610点 × 月1回

③ 検査・画像診断
   血液検査（CBC）: 21点
   HbA1c: 49点
   X線撮影: 68点（単純1枚）

④ 投薬・注射
   処方料: 42点（3種以上の内服薬: 29点に減算）
   処方箋料: 68点（院外処方の場合）
   薬剤料: 薬価から算出

⑤ 処置・手術
   各処置の所定点数を算定

⑥ 合計点数 × 10 = 医療費（円）
   患者負担割合（1割〜3割）を乗じて患者負担額を算出

⑦ 推定返戻金額
   エラーfindingがある患者の合計点数 × 10 = 返戻リスク金額
```

2.6 データモデル

```
@dataclass
class StepResult:
    """パイプライン各ステップの実行結果"""
    step_name: str
    success: bool
    findings: list[dict[str, str]]  # {"level": "error"|"warning"|"info", "message": ..., "patient_id": ...}
    details: dict[str, Any]
    error: str | None = None


@dataclass
class MedicalReceiptPipelineResult:
    """医療レセプト点検パイプライン全体の結果"""
    success: bool
    steps: list[StepResult]
    total_receipts: int                        # 処理レセプト数
    error_count: int                           # エラー件数
    warning_count: int                         # 警告件数
    estimated_rejection_amount: int            # 推定返戻金額（円）
    alerts_by_patient: dict[str, list[str]]    # 患者別アラート
    final_output: dict[str, Any]               # 最終出力（レポート含む）
    error: str | None = None
```

入力データ JSON構造

```
{
  "receipts": [
    {
      "patient_id": "P001",
      "patient_name": "山田太郎",
      "department": "内科",
      "month": "2026-03",
      "insurance_type": "社保",
      "copay_ratio": 0.3,
      "procedures": [
        {"name": "再診料", "count": 1, "points": 75},
        {"name": "特定疾患療養管理料", "count": 2, "points": 450},
        {"name": "外来管理加算", "count": 1, "points": 52}
      ],
      "medications": [
        {"name": "アムロジピン", "dose": "5mg", "days": 28},
        {"name": "メトホルミン", "dose": "500mg", "days": 28}
      ],
      "diseases": [
        {"name": "高血圧症", "code": "I10"},
        {"name": "2型糖尿病", "code": "E11"}
      ]
    }
  ]
}
```

2.7 法令・規格準拠ルール

法令	条文	適用箇所
健康保険法 第76条	療養の給付に関する費用の額の算定方法	全算定ルール
健康保険法施行規則 第47条	診療報酬の請求	請求期限（毎月10日）
保険医療機関及び保険医療養担当規則	第20条〜第23条	投薬日数制限・処方ルール
診療報酬の算定方法（厚生労働省告示）	別表第1（医科診療報酬点数表）	全点数ルール
薬機法 第14条	医薬品の承認	薬剤適応チェック
個人情報保護法 第2条第3項	要配慮個人情報	患者データの取り扱い
医療情報システムの安全管理に関するガイドライン（3省2ガイドライン）	第6.0版	システムセキュリティ

2.8 制約・前提条件

- 診療報酬点数表（約5,000項目）はゲノムJSONとして管理。2年ごとの改定時にJSON更新
- Phase 1ではCSV入力（レセコンからのエクスポート）のみ。ORCAとのAPI連携はPhase 2+
- 薬剤適応チェックは主要200薬剤のみ。網羅的チェックはPhase 2+で添付文書DBを構築
- DPC（包括払い）は対象外（19床以下のクリニックは出来高払いが大半）

2.9 独自価値

- **算定漏れの能動提案**: 「この患者に特定疾患療養管理料が算定可能です」
- **査定予測**: 支払基金の審査傾向を学習し、査定リスクをスコア化
- **歯科エンジンとの共通基盤**: 歯科で実証済みのチェックエンジンを医科に展開。開発コスト最小

---

3. ケアプラン作成支援（介護施設向け）

3.1 課題（金額換算の価値）

```
■ ケアマネジャーの工数削減
  ケアプラン作成: 1件あたり3-5時間 × 月30件 = 月90-150時間
  AI支援で50%削減 → 月45-75時間の削減
  時給2,000円 × 60時間 = 月12万円 = 年144万円

■ ケアプランの質向上
  標準化されたアセスメント→課題分析→目標設定
  個別ニーズの見落とし防止
  モニタリング漏れの防止

■ 合計効果：年150-200万円
  → BPO月4万円で ROI 3.1〜4.2倍
```

3.2 業務フロー

```
【新規利用者の場合】
  利用者情報・ADL評価入力
    → ★AI Step 1: assessment_analyzer（アセスメント分析）
    → ★AI Step 2: needs_extractor（ニーズ・課題抽出）
    → ★AI Step 3: goal_setter（長期・短期目標設定）
    → ★AI Step 4: service_planner（サービス内容計画）
    → ★AI Step 5: document_generator（第1表〜第7表生成）
    → ★AI Step 6: consistency_checker（整合性チェック）
    → ★AI Step 7: output_formatter（出力フォーマット変換）
    → ケアマネが確認・修正→利用者・家族に説明→同意取得

【モニタリング（月次）】
  サービス提供実績 + 利用者状態変化
    → ★AI: モニタリングシート自動生成
    → 目標達成度の自動評価
    → プラン見直し必要性の判定
```

3.3 パイプラインステップ定義

```
実装: workers/bpo/nursing/pipelines/care_plan_pipeline.py
クラス: CarePlanPipeline
```

Step 1: assessment_analyzer（アセスメント分析）

項目	内容
マイクロエージェント	`micro/assessment_analyzer`
入力	利用者基本情報、ADL評価（バーセルインデックス）、IADL評価、認知機能（HDS-R/MMSE）、家族状況
処理	課題分析表の自動生成（23項目のアセスメント領域を分析）
出力	課題分析結果（領域別の状態・課題・改善可能性）
法令根拠	指定居宅介護支援等の事業の人員及び運営に関する基準 第13条第7号

アセスメント23領域（厚生労働省通知 課題分析標準項目）

```
ASSESSMENT_DOMAINS = [
    # 基本情報
    {"id": 1, "name": "基本情報", "items": ["氏名", "性別", "生年月日", "住所"]},
    {"id": 2, "name": "生活状況", "items": ["家族構成", "主介護者", "住環境"]},
    {"id": 3, "name": "被保険者情報", "items": ["要介護度", "認定期間", "負担割合"]},
    # ADL
    {"id": 4, "name": "健康状態", "items": ["既往歴", "現病歴", "服薬状況", "主治医"]},
    {"id": 5, "name": "ADL", "items": ["寝返り", "起き上がり", "移乗", "歩行", "着脱衣", "入浴", "排泄"]},
    {"id": 6, "name": "IADL", "items": ["調理", "掃除", "洗濯", "買い物", "金銭管理", "服薬管理", "電話"]},
    {"id": 7, "name": "認知", "items": ["日常の意思決定", "記憶", "理解", "コミュニケーション"]},
    {"id": 8, "name": "コミュニケーション能力", "items": ["視力", "聴力", "意思伝達"]},
    {"id": 9, "name": "社会との関わり", "items": ["社会参加", "対人交流", "日中活動"]},
    {"id": 10, "name": "排尿・排便", "items": ["排尿", "排便", "おむつ使用"]},
    {"id": 11, "name": "褥瘡・皮膚の問題", "items": ["褥瘡有無", "皮膚状態"]},
    {"id": 12, "name": "口腔衛生", "items": ["歯の状態", "口腔清潔", "嚥下機能"]},
    {"id": 13, "name": "食事摂取", "items": ["食事形態", "摂取量", "栄養状態"]},
    {"id": 14, "name": "問題行動", "items": ["徘徊", "暴言暴行", "不潔行為", "異食"]},
    {"id": 15, "name": "介護力", "items": ["主介護者の健康状態", "介護負担度"]},
    {"id": 16, "name": "居住環境", "items": ["住宅改修の必要性", "バリアフリー状況"]},
    {"id": 17, "name": "特別な状況", "items": ["虐待リスク", "ターミナル", "認知症"]},
]

# バーセルインデックス（ADL評価）
BARTHEL_INDEX_ITEMS = {
    "食事": [0, 5, 10],          # 全介助/部分介助/自立
    "移乗": [0, 5, 10, 15],
    "整容": [0, 5],
    "トイレ動作": [0, 5, 10],
    "入浴": [0, 5],
    "歩行": [0, 5, 10, 15],
    "階段昇降": [0, 5, 10],
    "着替え": [0, 5, 10],
    "排便": [0, 5, 10],
    "排尿": [0, 5, 10],
}
# 合計100点満点。60点以下=中等度介助、40点以下=重度介助
```

Step 2: needs_extractor（ニーズ・課題抽出）

項目	内容
マイクロエージェント	`micro/needs_extractor`（LLM使用）
入力	Step 1のアセスメント結果
処理	LLMがアセスメントからニーズを抽出し、ICF（国際生活機能分類）に基づいて分類
出力	ニーズリスト（本人のニーズ/家族のニーズ/専門職が判断するニーズ）
LLMプロンプト	「以下のアセスメント結果から、ICFの枠組みに基づいて生活課題を抽出してください。心身機能・身体構造/活動/参加/環境因子/個人因子の5領域で整理してください。」

Step 3: goal_setter（長期・短期目標設定）

項目	内容
マイクロエージェント	`micro/goal_setter`（LLM使用）
入力	Step 2のニーズリスト
処理	ニーズごとに長期目標（6ヶ月）・短期目標（3ヶ月）を生成
出力	目標リスト（具体的・測定可能・達成可能・関連性・期限付き: SMART基準）
制約	目標は利用者の言葉で記述（「〜できるようになりたい」形式）

Step 4: service_planner（サービス内容計画）

項目	内容
マイクロエージェント	`micro/service_planner`（LLM使用）
入力	Step 3の目標リスト + 要介護度 + 区分支給限度額
処理	目標達成に必要なサービスの組み合わせを提案（限度額内に収める）
出力	週間サービス計画表（曜日×時間帯×サービス種類）
制約	区分支給限度基準額を超えないこと（超過分は自己負担として明示）

Step 5: document_generator（第1表〜第7表生成）

項目	内容
マイクロエージェント	`micro/document_generator`
入力	Step 1〜4の全データ
処理	居宅サービス計画書の各表を生成
出力	7種類の帳票データ（JSON→PDF/Excel変換）

ケアプラン帳票一覧

表	名称	内容	生成方式
第1表	居宅サービス計画書(1)	利用者の意向、総合的な援助の方針	LLM生成（ケアマネ確認必須）
第2表	居宅サービス計画書(2)	生活全般の解決すべき課題、目標、サービス内容	LLM生成
第3表	週間サービス計画表	曜日×時間帯のサービススケジュール	ルールベース生成
第4表	サービス担当者会議の要点	会議記録テンプレート	テンプレート生成
第5表	居宅介護支援経過	支援経過記録テンプレート	テンプレート生成
第6表	サービス利用票	月間サービス利用予定	ルールベース生成
第7表	サービス利用票別表	利用料の算出	計算ベース生成

Step 6: consistency_checker（整合性チェック）

項目	内容
マイクロエージェント	`micro/consistency_checker`
入力	全帳票データ
処理	アセスメント→課題→目標→サービスの一貫性チェック
出力	不整合リスト
チェック項目	課題に対応する目標があるか、目標に対応するサービスがあるか、限度額内か

Step 7: output_formatter（出力フォーマット変換）

項目	内容
マイクロエージェント	`micro/output_formatter`
入力	全帳票データ
処理	PDF/Excel/CSV出力
出力	ファイルバイナリ

3.4 データモデル

```
@dataclass
class CarePlanPipelineResult:
    success: bool
    user_id: str
    user_name: str
    care_level: int
    plan_period_start: str          # 計画開始日
    plan_period_end: str            # 計画終了日
    assessment_summary: dict        # アセスメント要約
    needs: list[dict]               # ニーズリスト
    goals: list[dict]               # 目標リスト（長期/短期）
    weekly_schedule: dict           # 週間サービス計画
    monthly_units: int              # 月間予定単位数
    limit_units: int                # 区分支給限度基準額
    utilization_rate: float         # 限度額利用率
    documents: dict[str, Any]       # 第1表〜第7表データ
    consistency_issues: list[str]   # 整合性問題
    steps_completed: list[str]
    error: str | None = None
```

3.5 法令根拠

法令	条文	適用箇所
介護保険法 第46条	居宅介護支援	ケアプラン作成の義務
指定居宅介護支援等の事業の人員及び運営に関する基準 第13条	具体的取扱方針	アセスメント→計画→実施→モニタリングの流れ
同基準 第13条第7号	課題分析の実施	アセスメント実施義務
同基準 第13条第13号	モニタリング	月1回以上の居宅訪問+モニタリング義務
厚生労働省通知 課題分析標準項目	23項目	アセスメントの標準項目

4. 勤怠・シフト管理（医療福祉共通）

4.1 課題（金額換算の価値）

```
■ シフト作成工数の削減
  看護師長・介護主任のシフト作成: 月20-40時間
  AI自動生成で80%削減 → 月16-32時間削減
  時給2,500円 × 24時間 = 月6万円 = 年72万円

■ 人員配置基準違反の防止
  配置基準違反 → 行政指導 → 最悪の場合指定取消
  人員基準欠如減算（70%減算）の防止: 年50-200万円の損失回避

■ 残業・超過勤務の最適化
  不適切なシフト → 残業発生 → 月10-30万円の追加人件費
  最適化で20%削減 = 年24-72万円

■ 合計効果：年146-344万円
  → BPO月4万円で ROI 3.0〜7.2倍
```

4.2 業務フロー

```
【月次（前月20日頃)】
 ★AI Step 1: staff_collector（スタッフ情報収集）
   → 希望休・有休・研修予定の取り込み
 ★AI Step 2: constraint_loader（制約条件ロード）
   → 人員配置基準・夜勤ルール・労基法制約のロード
 ★AI Step 3: shift_generator（シフト自動生成）
   → 制約充足ソルバーによるシフト案生成
 ★AI Step 4: compliance_checker（法令チェック）
   → 労基法・人員配置基準の充足確認
 ★AI Step 5: output_formatter（出力）
   → シフト表PDF/Excel + スタッフ通知

【日次】
 ★AI: 急な欠勤時の代替スタッフ提案
 ★AI: 翌日の配置基準充足プレチェック

【リアルタイム】
 ★AI: 勤怠打刻→実績との乖離検知→残業アラート
```

4.3 パイプラインステップ定義

```
実装: workers/bpo/common/pipelines/shift_pipeline.py
クラス: ShiftPipeline
```

Step 1〜5 概要

Step	名称	処理内容
1	staff_collector	スタッフ一覧、資格情報、希望休、有休残日数の取得
2	constraint_loader	人員配置基準、夜勤ルール、労基法制約のロード
3	shift_generator	制約充足問題としてシフト案を生成（ヒューリスティック最適化）
4	compliance_checker	全シフトパターンの法令チェック
5	output_formatter	シフト表出力 + 配置基準充足レポート

4.4 計算ロジック

夜勤ルール（介護施設）

```
NIGHT_SHIFT_RULES = {
    "72時間ルール": {
        "内容": "夜勤時間は2週間で72時間以内",
        "計算": "2週間の夜勤時間合計 ≤ 72時間",
        "法令": "看護師等の人材確保の促進に関する法律に基づく基本指針",
        "対象": "看護師",
    },
    "16時間ルール": {
        "内容": "夜勤は1回16時間以内（2交代制の場合）",
        "計算": "1回の夜勤勤務時間 ≤ 16時間",
        "法令": "労働基準法第32条（労働時間）",
        "対象": "全スタッフ",
    },
    "夜勤回数上限": {
        "内容": "月8回以内（推奨。法的上限ではないが行政指導基準）",
        "計算": "月間夜勤回数 ≤ 8回",
        "法令": "看護師等の人材確保の促進に関する法律 基本指針",
        "対象": "看護師・介護職員",
    },
    "夜勤明け休み": {
        "内容": "夜勤明けの翌日は休日とする（推奨）",
        "計算": "夜勤翌日 = 休日 or 遅番",
        "法令": "推奨事項（行政指導基準）",
    },
}
```

人員配置基準

```
STAFFING_REQUIREMENTS = {
    "特別養護老人ホーム": {
        "介護職員+看護職員": "入所者3名に対し1名以上",  # 3:1配置
        "看護職員": "入所者数に応じて1〜4名以上",
        "夜勤": {
            "25名以下": 1,  # 最低1名
            "26〜60名": 2,
            "61〜80名": 3,
            "81名以上": 4,
        },
        "法令": "指定介護老人福祉施設の人員、設備及び運営に関する基準 第2条",
    },
    "通所介護": {
        "介護職員": "利用者15名まで1名以上。15名超は5名ごとに+1名",
        "看護職員": "利用定員10名超で1名以上",
        "生活相談員": "サービス提供時間中1名以上",
        "法令": "指定居宅サービス等の事業の人員、設備及び運営に関する基準 第93条",
    },
    "訪問介護": {
        "サービス提供責任者": "利用者40名ごとに1名以上",
        "訪問介護員": "常勤換算2.5名以上",
        "法令": "指定居宅サービス等の事業の人員、設備及び運営に関する基準 第5条",
    },
    "クリニック（無床）": {
        "医師": "1名以上（管理者）",
        "看護師": "外来患者30名に対し1名以上",
        "法令": "医療法第21条",
    },
}

# 配置基準チェックロジック
def check_staffing(shift_date: str, facility: dict, assigned_staff: list) -> list[str]:
    """その日のシフトが配置基準を満たしているかチェック"""
    issues = []
    req = STAFFING_REQUIREMENTS[facility["type"]]

    if facility["type"] == "特別養護老人ホーム":
        residents = facility["resident_count"]
        required_care_nurse = residents / 3  # 3:1配置
        actual = len([s for s in assigned_staff if s["role"] in ("介護", "看護")])
        if actual < required_care_nurse:
            issues.append(
                f"人員配置基準違反: 介護+看護 {actual}名 < 必要{required_care_nurse:.0f}名"
                f"（入所者{residents}名 / 基準3:1）"
            )
    return issues
```

労働基準法の制約

```
LABOR_LAW_CONSTRAINTS = {
    "法定労働時間": {
        "内容": "1日8時間、週40時間",
        "法令": "労働基準法第32条",
    },
    "休日": {
        "内容": "週1日以上の休日",
        "法令": "労働基準法第35条",
    },
    "36協定上限": {
        "内容": "月45時間、年360時間（原則）",
        "法令": "労働基準法第36条",
    },
    "勤務間インターバル": {
        "内容": "終業から始業まで11時間以上（努力義務）",
        "法令": "労働時間等の設定の改善に関する特別措置法第2条",
    },
}
```

4.5 データモデル

```
@dataclass
class ShiftPipelineResult:
    success: bool
    period_start: str                       # シフト開始日
    period_end: str                         # シフト終了日
    facility_type: str                      # 施設種類
    shifts: list[dict]                      # シフト一覧（日付×スタッフ×勤務区分）
    staffing_check: list[dict]              # 配置基準充足状況（日別）
    night_shift_check: list[dict]           # 夜勤ルール充足状況
    labor_law_violations: list[str]         # 労基法違反
    optimization_score: float               # 最適化スコア（0-100）
    steps_completed: list[str]
    error: str | None = None
```

---

5. 記録・日誌AI（介護施設向け）

5.1 課題（金額換算の価値）

```
■ 記録作成工数の削減
  介護記録: 1人1日15-30分 × 職員10名 = 1日2.5-5時間
  月60-150時間の記録業務 → AI支援で60%削減 = 月36-90時間削減
  時給1,500円 × 60時間 = 月9万円 = 年108万円

■ 記録の質の標準化
  SOAP形式（主観/客観/評価/計画）の標準化
  利用者ごとの経時変化の自動検出

■ 合計効果：年110-160万円
  → BPO月2.5万円で ROI 3.7〜5.3倍
```

5.2 パイプラインステップ定義

```
実装: workers/bpo/nursing/pipelines/care_record_pipeline.py
クラス: CareRecordPipeline
```

Step 1〜7 概要

Step	名称	処理内容
1	input_collector	バイタルデータ、活動記録、口頭メモの取得
2	vital_analyzer	バイタル異常検知（閾値ベース+トレンド分析）
3	soap_generator	SOAP形式の介護記録自動生成（LLM使用）
4	change_detector	前回記録との状態変化検知
5	alert_generator	医師報告書・家族連絡書の自動生成（状態変化時）
6	daily_summary	日誌の自動生成（1日分の全利用者サマリー）
7	validator	必須記録項目の漏れチェック

5.3 計算ロジック

バイタル異常検知の閾値

```
VITAL_THRESHOLDS = {
    "体温": {
        "low": 35.0,       # 低体温
        "high": 37.5,      # 発熱
        "critical_high": 38.5,  # 高熱
        "unit": "℃",
    },
    "血圧_収縮期": {
        "low": 90,         # 低血圧
        "high": 140,       # 高血圧（ステージ1）
        "critical_high": 180,  # 高血圧緊急症
        "unit": "mmHg",
    },
    "血圧_拡張期": {
        "low": 60,
        "high": 90,
        "critical_high": 120,
        "unit": "mmHg",
    },
    "脈拍": {
        "low": 50,         # 徐脈
        "high": 100,       # 頻脈
        "critical_high": 120,
        "unit": "bpm",
    },
    "SpO2": {
        "low": 93,         # 低酸素（要注意）
        "critical_low": 90,  # 重度低酸素（医師報告必須）
        "unit": "%",
    },
    "血糖値": {
        "low": 70,         # 低血糖
        "critical_low": 54,  # 重度低血糖
        "high": 180,       # 高血糖（食後）
        "critical_high": 300,  # 重度高血糖
        "unit": "mg/dL",
    },
}

# 状態変化検知ロジック
def detect_vital_change(current: dict, history: list[dict]) -> list[str]:
    """直近7日間のバイタルトレンドから異常を検知"""
    alerts = []
    for key, thresholds in VITAL_THRESHOLDS.items():
        value = current.get(key)
        if value is None:
            continue
        # 閾値チェック
        if "critical_high" in thresholds and value >= thresholds["critical_high"]:
            alerts.append(f"【緊急】{key}: {value}{thresholds['unit']}（基準値超過）→ 医師報告必須")
        elif "high" in thresholds and value >= thresholds["high"]:
            alerts.append(f"【注意】{key}: {value}{thresholds['unit']}（高値）")
        # トレンド分析（3日連続上昇/下降）
        recent = [h.get(key) for h in history[-3:] if h.get(key)]
        if len(recent) >= 3 and all(recent[i] < recent[i+1] for i in range(len(recent)-1)):
            alerts.append(f"【トレンド】{key}: 3日連続上昇傾向")
    return alerts
```

SOAP形式の記録テンプレート（LLMプロンプト）

```
あなたは介護記録の専門家です。以下の情報からSOAP形式の介護記録を作成してください。

S（Subjective: 主観的情報）: 利用者の訴え、表情、気分
O（Objective: 客観的情報）: バイタルサイン、観察事項、ADL状況
A（Assessment: 評価）: S+Oからの状態評価、課題の変化
P（Plan: 計画）: 今後の対応方針、次回の観察ポイント

入力データ:
- バイタル: {vitals}
- 活動記録: {activities}
- 職員メモ: {staff_notes}
- 前回記録: {previous_record}
```

5.4 法令根拠

法令	条文	適用箇所
指定居宅サービス等の事業の人員、設備及び運営に関する基準 第19条	記録の整備	介護記録の作成義務（2年間保存）
同基準 第105条	通所介護の記録	サービス提供記録の作成義務
介護保険法施行規則 第140条の41	記録の保存	5年間の保存義務（条例による）

---

6. 服薬管理（医療福祉共通）

6.1 課題（金額換算の価値）

```
■ 服薬事故の防止
  投薬ミス（誤薬）: 年間報告件数2,000件以上（PMDA）
  1件の誤薬事故対応: 50-500万円（医療費+損害賠償+保険料上昇）
  防止率50%として: 年25-250万円のリスク回避

■ 飲み忘れ防止
  高齢者の服薬アドヒアランス: 50-70%（特に多剤処方患者）
  アドヒアランス向上 → 入院リスク低減

■ 薬剤相互作用チェック
  多剤処方（5剤以上）: 高齢者の40%以上
  相互作用による有害事象の防止

■ 合計効果：年50-300万円（リスク回避含む）
  → BPO月2.5万円で ROI 1.7〜10倍
```

6.2 パイプラインステップ定義

```
実装: workers/bpo/common/pipelines/medication_pipeline.py
クラス: MedicationManagementPipeline
```

Step	名称	処理内容
1	prescription_parser	処方データの取得・正規化（処方箋OCR or データ連携）
2	interaction_checker	薬剤相互作用チェック（禁忌・注意の判定）
3	schedule_generator	服薬スケジュール生成（朝食後/昼食後/夕食後/就寝前）
4	adherence_monitor	服薬アドヒアランス監視（飲み忘れ検出+アラート）
5	side_effect_monitor	副作用モニタリング（バイタル変動+症状との関連分析）
6	report_generator	服薬管理レポート生成（医師・薬剤師向け）
7	validator	最終バリデーション

6.3 計算ロジック

薬剤相互作用チェック

```
# 相互作用レベル
INTERACTION_LEVELS = {
    "禁忌": "CONTRAINDICATED",     # 絶対に併用不可
    "原則禁忌": "GENERALLY_AVOID",  # 原則併用不可（やむを得ない場合を除く）
    "併用注意": "CAUTION",          # 併用可能だが注意が必要
    "併用注意（減量）": "DOSE_ADJUST",  # 減量が必要
}

# 主要な相互作用ルール（一部抜粋）
INTERACTION_RULES = [
    {
        "drug_a": "ワルファリン",
        "drug_b_group": ["NSAIDs", "アスピリン"],
        "level": "GENERALLY_AVOID",
        "effect": "出血リスク増大",
        "mechanism": "抗凝固作用の増強+胃粘膜障害",
        "法令": "医薬品添付文書（薬機法第52条）",
    },
    {
        "drug_a": "ACE阻害薬",
        "drug_b": "カリウム保持性利尿薬",
        "level": "CAUTION",
        "effect": "高カリウム血症リスク",
        "mechanism": "カリウム排泄抑制の相加作用",
    },
    {
        "drug_a": "メトホルミン",
        "drug_b": "ヨード造影剤",
        "level": "CONTRAINDICATED",
        "effect": "乳酸アシドーシスリスク",
        "mechanism": "腎機能低下時のメトホルミン蓄積",
    },
    {
        "drug_a": "SSRI",
        "drug_b": "MAO阻害薬",
        "level": "CONTRAINDICATED",
        "effect": "セロトニン症候群",
        "mechanism": "セロトニン作用の過剰増強",
    },
]

# ポリファーマシー（多剤処方）チェック
def check_polypharmacy(medications: list[dict]) -> list[str]:
    """多剤処方の警告"""
    warnings = []
    if len(medications) >= 6:
        warnings.append(
            f"ポリファーマシー警告: {len(medications)}剤処方。"
            "有害事象リスクが高まります。処方見直しを推奨します。"
            "（高齢者の医薬品適正使用の指針: 厚生労働省）"
        )
    # 同効薬の重複チェック
    drug_classes = [m.get("class", "") for m in medications]
    duplicates = [c for c in set(drug_classes) if drug_classes.count(c) > 1 and c]
    for dup in duplicates:
        warnings.append(f"同効薬重複: {dup}が2剤以上処方されています")
    return warnings
```

6.4 法令根拠

法令	条文	適用箇所
薬機法 第52条	添付文書の記載事項	相互作用チェックの根拠
医薬品の臨床試験の実施の基準に関する省令	GCP省令	安全性情報の取り扱い
高齢者の医薬品適正使用の指針（厚生労働省）	総論編・各論編	ポリファーマシー対策
介護保険施設における医薬品の使用指針	服薬管理の基準	施設での服薬管理ルール
個人情報保護法 第2条第3項	要配慮個人情報	処方データの取り扱い

---

7. 監査・実地指導準備（医療福祉共通）

7.1 課題（金額換算の価値）

```
■ 監査準備工数の削減
  実地指導（行政監査）: 2-3年に1回
  準備工数: 延べ40-80時間（管理者+事務員）
  AI自動チェックで70%削減 → 28-56時間削減
  時給2,000円 × 40時間 = 8万円/回 = 年4万円

■ 指摘事項の未然防止
  行政指導→改善命令→最悪の場合指定取消
  過去の指摘事項を事前にチェックし、日常的に是正
  指定取消リスクの回避: 計測不能（事業継続リスク）

■ 返還金リスクの回避
  不適切な加算算定の指摘→過去に遡って返還
  返還金: 数十万〜数千万円
  事前チェックで返還リスクを最小化

■ 合計効果：年50-500万円（リスク回避含む）
  → BPO月2.5万円で ROI 1.7〜16.7倍
```

7.2 パイプラインステップ定義

```
実装: workers/bpo/common/pipelines/audit_prep_pipeline.py
クラス: AuditPrepPipeline
```

Step	名称	処理内容
1	document_scanner	必要書類の一覧化と存在チェック
2	checklist_generator	指導監査チェックリスト自動生成（指定基準別）
3	gap_analyzer	基準と現状のギャップ分析
4	past_issue_matcher	過去の指摘事項との照合
5	remediation_planner	是正計画の自動生成
6	report_generator	監査準備レポート出力
7	validator	最終バリデーション

7.3 計算ロジック

指導監査チェックリスト（介護施設の場合）

```
AUDIT_CHECKLIST_NURSING = {
    "人員基準": {
        "items": [
            {"id": "P001", "name": "管理者の専従・常勤要件", "基準": "基準省令第6条"},
            {"id": "P002", "name": "介護職員の配置基準充足", "基準": "基準省令第93条"},
            {"id": "P003", "name": "看護職員の配置基準充足", "基準": "基準省令第93条"},
            {"id": "P004", "name": "生活相談員の配置", "基準": "基準省令第93条"},
            {"id": "P005", "name": "機能訓練指導員の配置", "基準": "基準省令第93条"},
        ],
        "確認書類": ["勤務表（直近6ヶ月）", "雇用契約書", "資格証明書"],
    },
    "設備基準": {
        "items": [
            {"id": "E001", "name": "食堂・機能訓練室の面積基準", "基準": "基準省令第95条"},
            {"id": "E002", "name": "静養室の設置", "基準": "基準省令第95条"},
            {"id": "E003", "name": "消防設備の適切な設置", "基準": "消防法第17条"},
        ],
        "確認書類": ["平面図", "消防設備点検報告書"],
    },
    "運営基準": {
        "items": [
            {"id": "O001", "name": "重要事項説明書の交付・同意", "基準": "基準省令第8条"},
            {"id": "O002", "name": "サービス提供記録の作成・保管", "基準": "基準省令第19条"},
            {"id": "O003", "name": "苦情処理体制の整備", "基準": "基準省令第36条"},
            {"id": "O004", "name": "事故報告体制の整備", "基準": "基準省令第37条"},
            {"id": "O005", "name": "身体拘束の適正化委員会の開催", "基準": "基準省令第11条第5項"},
            {"id": "O006", "name": "虐待防止委員会の開催", "基準": "基準省令第37条の2"},
            {"id": "O007", "name": "感染症対策委員会の開催", "基準": "基準省令第27条"},
            {"id": "O008", "name": "ケアプランに基づくサービス提供", "基準": "介護保険法第41条第6項"},
        ],
        "確認書類": [
            "重要事項説明書", "契約書", "サービス提供記録",
            "苦情対応記録", "事故報告書", "委員会議事録",
        ],
    },
    "加算関連": {
        "items": [
            {"id": "A001", "name": "処遇改善加算の計画書提出", "基準": "告示19号"},
            {"id": "A002", "name": "処遇改善加算の実績報告", "基準": "告示19号"},
            {"id": "A003", "name": "特定事業所加算の要件充足", "基準": "告示19号別表1注7"},
            {"id": "A004", "name": "加算算定に係る各種記録の整備", "基準": "各加算の算定要件"},
        ],
        "確認書類": [
            "処遇改善計画書", "賃金台帳", "研修記録",
            "会議録", "個別機能訓練計画",
        ],
    },
}

# 書類欠損チェックロジック
def check_document_gaps(
    facility_documents: list[str],
    required_documents: list[str],
) -> list[dict]:
    """必要書類の欠損を検出"""
    gaps = []
    for doc in required_documents:
        if doc not in facility_documents:
            gaps.append({
                "document": doc,
                "status": "MISSING",
                "risk_level": "HIGH",
                "recommendation": f"「{doc}」が見つかりません。実地指導までに準備してください。",
```

```
            })
    return gaps
```

7.4 法令根拠

法令	条文	適用箇所
介護保険法 第76条	報告等	行政の指導監査権限
介護保険法 第77条	勧告・命令・指定取消	基準違反時の処分
指定居宅サービス等の事業の人員、設備及び運営に関する基準	全般	チェックリストの根拠
医療法 第25条	立入検査	クリニックの行政検査
消防法 第17条	消防設備の設置義務	設備基準チェック

---

8. 請求・入金管理（医療福祉共通）

8.1 課題（金額換算の価値）

```
■ 未収金の回収率向上
  介護施設の自己負担未収金: 月間利用者100名で月5-15万円
  回収率10%改善 → 年6-18万円

■ 公費適用判定の正確化
  生活保護・自立支援医療・特定疾患等の適用漏れ → 本来不要な請求
  適用漏れの防止: 年12-36万円

■ 事務工数の削減
  請求書作成+入金確認+督促: 月20-30時間
  AI自動化で60%削減 → 月12-18時間削減
  時給1,500円 × 15時間 = 月2.25万円 = 年27万円

■ 合計効果：年45-81万円
  → BPO月2.5万円で ROI 1.5〜2.7倍
```

8.2 パイプラインステップ定義

```
実装: workers/bpo/common/pipelines/billing_collection_pipeline.py
クラス: BillingCollectionPipeline
```

Step	名称	処理内容
1	billing_calculator	自己負担金の計算（負担割合×保険給付対象外の算定）
2	public_expense_checker	公費負担制度の適用判定
3	invoice_generator	請求書の自動生成（PDF）
4	payment_tracker	入金管理（消込処理）
5	dunning_manager	未収金の督促フロー管理
6	aging_reporter	滞納エイジングレポート生成
7	validator	最終バリデーション

8.3 計算ロジック

自己負担金の計算

```python
# 利用者負担割合の判定
COPAY_RATIOS = {
    "一般": 0.1,           # 1割負担（大多数の高齢者）
    "一定以上所得者": 0.2,  # 2割負担（年金収入280万円以上）
    "現役並み所得者": 0.3,  # 3割負担（年金収入340万円以上）
}

# 高額介護サービス費（月額上限）
HIGH_COST_LIMITS = {
    "生活保護受給者": 15_000,         # 月15,000円
    "市町村民税非課税（年金80万以下）": 15_000,
    "市町村民税非課税": 24_600,
    "一般": 44_400,
    "現役並み所得者I": 44_400,
    "現役並み所得者II": 93_000,
    "現役並み所得者III": 140_100,
}

# 公費負担制度
PUBLIC_EXPENSE_PROGRAMS = {
    "生活保護（介護扶助）": {
        "法令": "生活保護法第15条の2",
        "内容": "利用者負担額の全額を公費負担",
        "判定": "生活保護受給者証の番号確認",
        "負担者番号": "12xxxxxx",
    },
    "自立支援医療（精神通院）": {
        "法令": "障害者総合支援法第58条",
        "内容": "自己負担を1割に軽減+月額上限設定",
        "判定": "自立支援医療受給者証の番号確認",
    },
    "特定疾患医療（難病）": {
        "法令": "難病の患者に対する医療等に関する法律第5条",
        "内容": "自己負担を2割に軽減+月額上限設定",
        "判定": "特定医療費受給者証の番号確認",
    },
    "被爆者援護": {
        "法令": "原子爆弾被爆者に対する援護に関する法律第10条",
        "内容": "医療費の全額公費負担",
        "判定": "被爆者健康手帳の番号確認",
    },
}

# 自己負担金計算
def calculate_copay(
    total_amount: int,
    copay_ratio: float,
    income_class: str,
    public_expense: str | None,
) -> dict:
    """自己負担金の計算"""
    base_copay = int(total_amount * copay_ratio)

    # 公費適用
    if public_expense == "生活保護（介護扶助）":
        return {"copay": 0, "public_expense_amount": base_copay, "reason": "生活保護法第15条の2"}

    # 高額介護サービス費の適用
    monthly_limit = HIGH_COST_LIMITS.get(income_class, 44_400)
    if base_copay > monthly_limit:
        actual_copay = monthly_limit
        high_cost_benefit = base_copay - monthly_limit
    else:
```

```
        actual_copay = base_copay
        high_cost_benefit = 0

    return {
        "copay": actual_copay,
        "high_cost_benefit": high_cost_benefit,
        "public_expense_amount": 0,
    }
```

督促フロー

```
DUNNING_FLOW = [
    {
        "days_overdue": 0,
        "action": "請求書発行",
        "method": "郵送 or 手渡し",
    },
    {
        "days_overdue": 14,
        "action": "リマインド通知",
        "method": "電話 or SMS",
        "template": "お支払い期日を過ぎておりますので、ご確認をお願いいたします。",
    },
    {
        "days_overdue": 30,
        "action": "督促状送付",
        "method": "郵送（内容証明ではない）",
        "template": "前回のご請求について、まだご入金が確認できておりません。",
    },
    {
        "days_overdue": 60,
        "action": "管理者面談設定",
        "method": "電話 → 面談",
        "note": "支払い計画の相談。分割払い等の提案",
    },
    {
        "days_overdue": 90,
        "action": "内容証明郵便",
        "method": "内容証明郵便",
        "note": "法的措置の可能性を示唆",
    },
]
```

8.4 法令根拠

法令	条文	適用箇所
介護保険法 第41条第4項	利用者負担	自己負担の根拠
介護保険法 第51条	高額介護サービス費	月額上限の適用
生活保護法 第15条の2	介護扶助	公費負担判定
障害者総合支援法 第58条	自立支援医療	公費負担判定
難病の患者に対する医療等に関する法律 第5条	特定医療費	公費負担判定
民法 第166条	債権の消滅時効	未収金の時効管理（5年）
電子帳簿保存法	電子データの保存要件	請求書の電子保存

データモデル（共通）

DB設計

```
-- 介護利用者マスタ
CREATE TABLE care_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    insured_number TEXT NOT NULL,             -- 被保険者番号
    last_name TEXT NOT NULL,
    first_name TEXT NOT NULL,
    birth_date DATE NOT NULL,
    care_level INTEGER,                       -- 要介護度（1〜5）
    care_level_valid_from DATE,               -- 認定有効期間開始日
    care_level_valid_to DATE,                 -- 認定有効期間終了日
    copay_ratio DECIMAL(2,1) DEFAULT 0.1,     -- 負担割合
    income_class TEXT DEFAULT '一般',          -- 所得区分
    public_expense_code TEXT,                  -- 公費負担者番号
    insurer_number TEXT,                       -- 保険者番号
    primary_doctor TEXT,                       -- 主治医
    medications JSONB DEFAULT '[]',            -- 服薬情報
    allergies JSONB DEFAULT '[]',              -- アレルギー情報
    status TEXT DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 患者マスタ（クリニック）
CREATE TABLE clinic_patients (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    patient_number TEXT NOT NULL,
    last_name TEXT NOT NULL,
    first_name TEXT NOT NULL,
    birth_date DATE NOT NULL,
    insurance_type TEXT NOT NULL,              -- 社保/国保/後期高齢者
    insurance_number TEXT,
    copay_ratio DECIMAL(2,1) DEFAULT 0.3,
    chronic_diseases JSONB DEFAULT '[]',       -- 慢性疾患リスト
    medications JSONB DEFAULT '[]',
    status TEXT DEFAULT 'active',
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- サービス提供実績
CREATE TABLE care_service_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    care_user_id UUID NOT NULL REFERENCES care_users(id),
    service_type TEXT NOT NULL,
    service_code TEXT NOT NULL,
    service_date DATE NOT NULL,
    service_hours DECIMAL(4,2),
    units INTEGER NOT NULL,
    addition_codes JSONB DEFAULT '[]',
    staff_id UUID,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT now()
);
```

```sql
-- 介護報酬請求
CREATE TABLE care_billing_claims (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    period_year INTEGER NOT NULL,
    period_month INTEGER NOT NULL,
    facility_code TEXT NOT NULL,
    total_units INTEGER NOT NULL,
    total_amount INTEGER NOT NULL,
    insurance_amount INTEGER NOT NULL,
    copay_amount INTEGER NOT NULL,
    status TEXT DEFAULT 'draft',              -- draft/submitted/accepted/rejected/corrected
    rejection_reason TEXT,
    submitted_at TIMESTAMPTZ,
    result_received_at TIMESTAMPTZ,
    pipeline_result JSONB,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 未収金管理
CREATE TABLE receivables (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    user_id UUID NOT NULL,                   -- care_users.id or clinic_patients.id
    user_type TEXT NOT NULL,                  -- care_user / clinic_patient
    invoice_date DATE NOT NULL,
    due_date DATE NOT NULL,
    amount INTEGER NOT NULL,
    paid_amount INTEGER DEFAULT 0,
    status TEXT DEFAULT 'unpaid',             -- unpaid/partial/paid/overdue/written_off
    dunning_step INTEGER DEFAULT 0,
    last_dunning_date DATE,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- RLS
ALTER TABLE care_users ENABLE ROW LEVEL SECURITY;
ALTER TABLE clinic_patients ENABLE ROW LEVEL SECURITY;
ALTER TABLE care_service_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE care_billing_claims ENABLE ROW LEVEL SECURITY;
ALTER TABLE receivables ENABLE ROW LEVEL SECURITY;
```

セキュリティ・コンプライアンス

要配慮個人情報の取り扱い

医療・福祉データは個人情報保護法第2条第3項に定める「要配慮個人情報」に該当する。

```
保護対象:
  - 要介護度、ADL評価、認知機能評価
  - 傷病名、治療歴、処方薬
  - 家族構成、所得区分、生活保護受給状況
  - バイタルデータ、介護記録

必須対策:
  ① テナント分離（company_idベースのRLS）→ 全テーブルに適用
  ② 全アクセスをaudit_logsに記録（誰が、いつ、何を、閲覧/編集）
  ③ 3省2ガイドライン準拠（医療情報システムの安全管理に関するガイドライン 第6.0版）
  ④ PII正規表現マスキング（被保険者番号、保険者番号）
  ⑤ 暗号化: Supabase nativeのAES-256（Phase 1）→ GCP KMS（Enterprise）
```

法令準拠一覧

法令	管轄	主な要件
個人情報保護法	個人情報保護委員会	要配慮個人情報の取得同意、安全管理措置
介護保険法	厚生労働省	介護報酬の算定基準、事業者の指定基準
健康保険法	厚生労働省	診療報酬の算定方法、保険医療機関の基準
医療法	厚生労働省	診療所の開設基準、安全管理
薬機法	厚生労働省	医薬品の適正使用
労働基準法	厚生労働省	勤務時間、休日、夜勤
3省2ガイドライン	厚生労働省/経産省/総務省	医療情報システムの安全管理
電子帳簿保存法	国税庁	請求書等の電子保存

競合環境

サービス	月額	強み	弱み
カイポケ（SMS）	¥2-5万	介護ソフト最大手	ツールのみ。AI判断なし
ほのぼのNEXT（NDソフト）	¥3-8万	老舗。大規模施設に強い	UIが古い。AI活用弱い
カナミック	¥2-5万	クラウド型。中小に強い	機能限定的
ORCA（日医標準）	無料〜	レセコン普及率高い	点検機能は最低限
AI-SORA等AIレセプト	¥3-10万	AI査定予測	単機能

シャチョツーの差別化:

1. **判断するAI**: 既存は「入力ツール」。シャチョツーは「チェック+提案+生成」まで行うAIエージェント
2. **8モジュール一気通貫**: 請求+ケアプラン+シフト+記録+服薬+監査+入金を統合
3. **加算の能動提案**: 「取れるのに取っていない加算」を見つけて提案する
4. **歯科エンジンの横展開**: 実証済みのチェックエンジンを最小コストで医科・介護に展開
5. **学習する算定DB**: 使うほど事業所固有のパターンを学習し精度が上がる